



Autor: Andrés Xavier Rubio Proaño
Paralelo: 01

Docente: Andrés Xavier Rubio Proaño
Fecha: 5 de octubre de 2026

Ejemplo ilustrativo del formato esperado: corto, con lo que hiciste y lo que respondió tu terminal.

Objetivo

Dejar listo el entorno del curso (Ubuntu 26.04, gcc, make, man) y comprobar cómo informa el curso los errores de las llamadas al sistema. Equipo: Linux 7.0.0, x86_64, 28 procesadores.

TD0 · 1 Páginas del manual y secciones

Qué hice. Busqué printf por nombre exacto con `man -k`.

```
$ man -k '^printf$'
printf (1)      - Print output based off of the format string and procee...
printf (3)      - formatted output conversion
```

Conclusión. `printf(1)` es el comando del shell y `printf(3)` la función de C (glibc). La sección 3p (el contrato POSIX, portable) aparece al instalar `manpages-posix-dev`: para programar para varios sistemas, la 3p; para depurar en Linux, la 3.

TD0 · 2 El esqueleto de programa

Qué hice. Copié `td0/skeleton.c`, el punto de partida de cada ejercicio, y lo compilé con las opciones del curso. Los `(void)` evitan que `-Werror` falle por parámetros sin usar.

```
1 int main(int argc, char *argv[])
2 {
3     (void)argc;    /* silences -Wunused-parameter until you use them */
4     (void)argv;
5     return EXIT_SUCCESS;
6 }
```

```
$ gcc -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -Icommon td0/skeleton.c -lbin
→ -lutils -o skeleton
$ ./skeleton; echo "estado: $?"
estado: 0
```

TD0 · 3 La biblioteca libutils y exit_if

Qué hice. `errors.c` abre un archivo y, si `open` devuelve `-1`, llama a `exit_if`, que imprime el mensaje de `errno` con `perror` y termina con estado 1.

```
1 const char *path = argc > 1 ? argv[1] : "missing.txt";
2 int fd = open(path, O_RDONLY);
3
4 exit_if(fd == -1, path);          /* prefix = the file name */
5 printf("%s opened as file descriptor %d\n", path, fd);
```

```
$ ./errors
missing.txt: No such file or directory
[exit status 1]
$ ./errors notes.txt
notes.txt opened as file descriptor 3
```

Conclusión. El error llega como `ENOENT` y `perror` lo traduce. Al abrir bien, el descriptor es 3: el 0, el 1 y el 2 ya son la entrada, la salida y el error estándar. `exit_if` va justo después de la llamada, porque otra llamada intermedia podría cambiar `errno`.

TD0 · 4 Compilar con make

Qué hice. Leí las reglas del Makefile del curso que construyen la biblioteca:

```
1 CFLAGS += -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -Icommon
2 LDLIBS += -Lbin -lutils -lm
3
4 bin/libutils.a: common/utils.c common/utils.h
5     $(CC) $(CFLAGS) -c common/utils.c -o bin/utils.o
6     $(AR) rcs $@ bin/utils.o
```

Conclusión. `libutils.a` se rehace solo si cambian `utils.c` o `utils.h`; cada ejercicio se enlaza con `-lutils`. Las variantes de un mismo programa (`buffer_1...`) se compilan con macros `-D`.

TD0 · 5 Explora tu sistema operativo

```
$ uname -srvm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2
→ | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) + 16.571s
→ (userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

Conclusión. Un solo núcleo (Linux 7.0) sirve a la distribución; el primer proceso (PID 1) es `systemd`; hay 237 módulos cargados aunque Linux es monolítico; el núcleo es la etapa más corta del arranque (3,3 s de 32 s).

Reflexión

Lo más útil fue ver que cada error de una llamada al sistema deja su causa en `errno`: es la base para diagnosticar el entorno del PIS (permisos, rutas, puertos) con mensajes claros.



Declaración de uso de IA generativa

Herramienta: el asistente de Prism (ChatGPT).

Para qué: dar formato a las salidas en el entorno `terminal` y revisar la redacción.

Cómo lo verifiqué: ejecuté cada comando en mi terminal y copié su salida tal cual; compilé `skeleton.c` y `errors.c` con `-Werror` sin advertencias.